计算机体系结构习题实验报告

多周期处理器、流水线、精确异常、分支预测与 Cache

2026 年 10 月 25 日

目录

一、题目来源与实验范围 **2**

二、实验目标与统一约定 **2**

2.1 实验目标 2

2.2 原程序与动态执行次数 3

2.3 本报告采用的参考模型 3

三、第 1 题：多周期处理器 **4**

四、第 2 题：无前递的单发射五级静态流水线 **5**

4.1 数据停顿 5

4.2 控制停顿与总周期 5

五、第 3 题：含前递的单发射五级静态流水线 **6**

六、第 4 题：增加精确异常支持 **6**

6.1 精确异常的状态要求 6

6.2 修改后的处理器结构 7

6.3 异常处理流程 8

6.4 为什么 store 和中断需要单独处理 8

七、第 5 题：含前递的双发射五级静态流水线 **9**

八、第 6 题：数据相关与解决方法 **9**

8.1 三种寄存器数据相关 9

8.2 静态流水线的处理 10

8.3 带寄存器重命名的动态流水线 10

九、第 7 题：D 级永远预测跳转 **10**

9.1 预测与验证流程 10
9.2 最后一次误判为什么不能机械再加罚时 11
9.3 计时终点 12

十、第 8 题：三重循环的 BHT 预测准确率 **12**
10.1 必须明确的统计约定 12
10.2 一位 BHT 13
10.3 两位饱和计数器 BHT 13
10.4 为什么有些答案会不同 14

十一、第 9 题：Cache 地址字段划分 **14**

十二、第 10 题：Cache 命中率与主频的性能权衡 **15**

十三、实验复核与结果汇总 **16**
13.1 可复现方法 16
13.2 本次执行的有效检查 16
13.3 答案汇总 16

一、题目来源与实验范围

本报告依据仓库 `zxymao/Chatgpt` 的三张原始图片完成，共涉及 10 道习题：

- 体系结构 1.png：第 1 题题干。
- 体系结构 2.png：一张标为“图 9.26 龙芯 3A3000 流水线示意图”的结构图。
- 体系结构 3.png：程序及第 2—10 题。

补充教材原图为：图 9.6、图 9.4、图 9.13、图 9.18。以下按这些图中的控制节拍、前递位置和双发射相关约束分析。

本报告属于**习题解析与教学模型仿真实验**：通过 Python 逐周期执行模型和 BHT 状态机复核推导；精确异常部分给出逻辑设计。未进行真实 CPU 测时、RTL 实现或硬件验证，也未虚构实测结果。

二、实验目标与统一约定

2.1 实验目标

1. 区分多周期执行与流水线执行的时钟周期计算。
2. 分析 RAW 数据相关、前递、控制冲刷对时序的影响。
3. 设计支持精确异常的顺序提交机制。

4. 通过状态机计算分支预测准确率，考虑冷启动影响。
5. 完成 Cache 地址划分和考虑主频变化的性能比较。

2.2 原程序与动态执行次数

```
        addi.w  t0, zero, 100
LOOP:
        addi.w  t0, t0, -1
        bnez    t0, LOOP
```

记初始化指令为 I，第 i 次循环的减一指令为 D_i，条件分支为 B_i。

初始化后 `t0=100`，依次减到 0。因此：

$$N_I = 1, \qquad N_D = 100, \qquad N_B = 100,$$

$$N_{\text{inst}} = 201, \qquad N_T = 99, \qquad N_N = 1.$$

前 99 个分支跳转，第 100 个分支不跳转。不是 101 次循环，也不是 100 次跳转。

2.3 本报告采用的参考模型

项目	采用的约定
多周期	图 9.4 五分频控制，每条指令固定经过 F、D、E、M、W 五拍；指令之间不重叠
五级流水线	F 取指、D 译码/读寄存器、E 执行、M 访存、W 写回；指令顺序执行、顺序提交
数据相关	无前递时，生产者在 W 写入后，消费者才能在 D 读到；允许同周期先写后读
前递	图 9.13 将运算器输出、R3、R4 送至 R2 前的操作数选择器，使相邻 ALU 指令不需数据停顿
分支	在 E 级末确定方向和目标；第 2、3、5 题默认顺序取指，即预测不跳转；实际跳转冲刷 F、D
双发射	最多每周期发射 2 条；同组 RAW 指令不能同时发射；跨组具有前递；没有循环展开或指令重排

项目	采用的约定
第 7 题预测	D 级识别分支并算出目标，下一周期按跳转取指；E 级验证；错误时冲刷年轻指令
流水线计时终点	为统一口径，计至最后一条分支通过 W 级；即使分支不写寄存器，也在图中保留 M、W
理想环境	无 Cache/TLB 不命中、无结构冲突或异常；不含程序段之后的其他有效指令

时空图中 D* 表示不能发射而停留在 D；F* 表示因后级停顿而保持取指结果。空白格表示该条指令不占用此级。图只展示真实路径上的 $I, D_1, B_1, D_2, B_2, D_3, B_3$；被冲刷的错误路径不另列行。前三次循环属于原程序的前缀，B_3 仍然跳转。

三、第 1 题：多周期处理器

图 9.4 通过五分频控制依次完成取指、译码、执行、访存和写回。每条指令占用全部五个控制节拍；即使某指令不访存或不写通用寄存器，相应节拍也保留。因此：

$$\boxed{C_1 = (1 + 100 + 100) \times 5 = 1005.}$$

前三次循环的具体时段如下。

指令	F	D	E	M	W
I	1	2	3	4	5
D_1	6	7	8	9	10
B_1	11	12	13	14	15
D_2	16	17	18	19	20
B_2	21	22	23	24	25
D_3	26	27	28	29	30
B_3	31	32	33	34	35

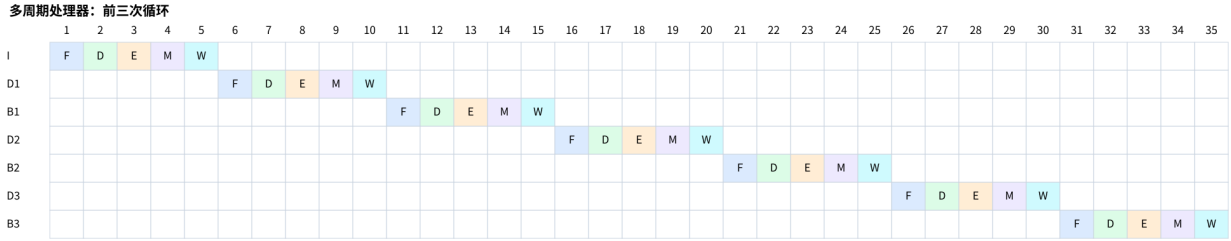


图 1: 第 1 题：多周期处理器前三次循环时空图

前三次循环加初始化共 7 条指令，占用 35 个周期。这里不能将不使用的访存或写回阶段直接从五分频时序中删去。

四、第 2 题：无前递的单发射五级静态流水线

4.1 数据停顿

`I → D1` 存在 RAW 相关；每个 `Di → Bi` 也存在 RAW 相关，共 101 对。

例如 I 在第 5 周期 W 写入 `t0`，D_1 原计划第 3 周期读寄存器，必须等到第 5 周期，增加 2 个停顿。每个分支等待紧邻减一指令的结果，也增加 2 个停顿。

$$C_{\text{RAW}} = 2 + 100 \times 2 = 202.$$

跨循环的减一指令也依赖前一次减一，但等待分支完成和重定向的间隔已经覆盖其结果产生时间，不额外增加停顿。

4.2 控制停顿与总周期

分支在 E 级末判定，跳转后下一周期才取目标指令，相对于理想相邻取指损失 2 周期。共 99 次跳转：

$$C_{\text{ctrl}} = 99 \times 2 = 198.$$

理想五级流水线执行 201 条指令为 $201 + 4 = 205$ 周期，因此：

$$\boxed{C_2 = 205 + 202 + 198 = 605.}$$

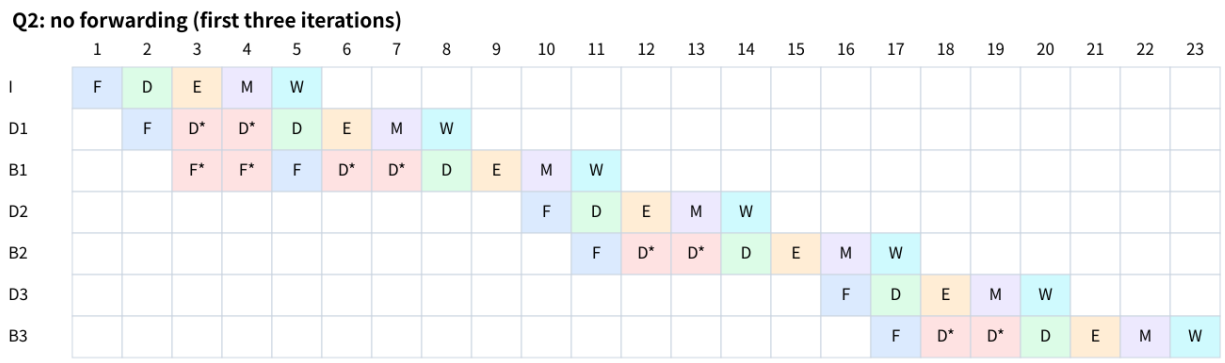


图 2: 第 2 题：无前递流水线前三次循环时空图

关键时刻：B_1 的 E 为第 9 周期，D_2 的 F 为第 10 周期；B_2 的 E 为第 15 周期，D_3 的 F 为第 16 周期。最终 B_{100} 在第 603 周期 E、第 605 周期 W。

五、第 3 题：含前递的单发射五级静态流水线

本程序没有 load 指令。每条 `addi.w` 的结果在 E 级末可用，下一条指令可以在自己的 E 级通过前递得到结果。`I → D1` 和 `Di → Bi` 的 RAW 相关均不需要数据停顿。

分支仍在 E 判定，前 99 次跳转各损失 2 周期：

$$\boxed{C_3 = 201 + 4 + 99 \times 2 = 403.}$$

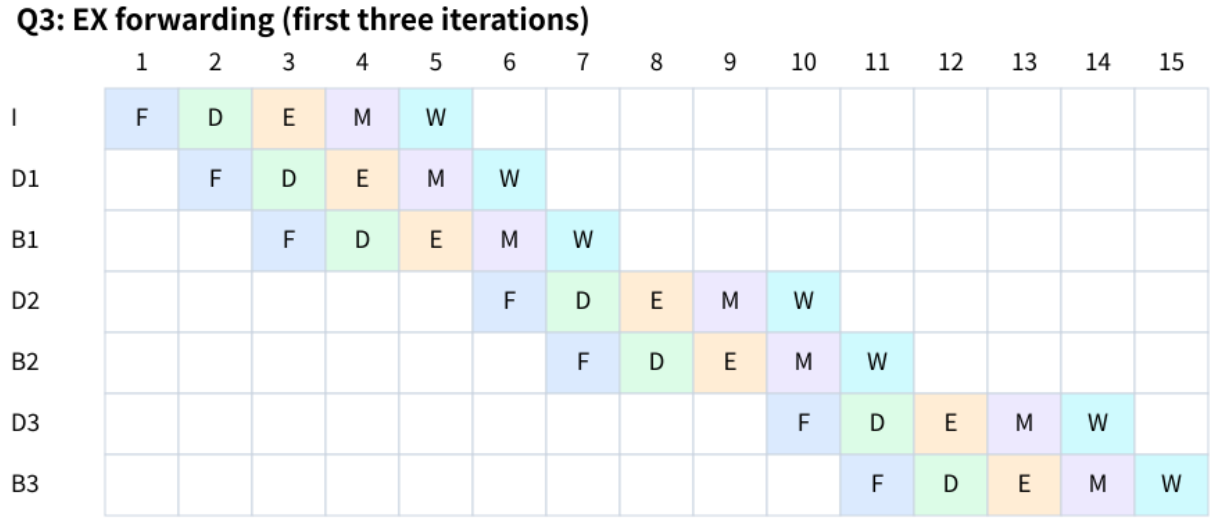


图 3: 第 3 题：含前递流水线前三次循环时空图

例如 D_1 在第 4 周期 E 计算出 99，B_1 在第 5 周期 E 通过前递用 99 判断跳转，D_2 在第 6 周期取指。最终 B_{100} 在第 401 周期 E、第 403 周期 W。

图 9.13 的前递选择器位于 R2 前，运算器输出、R3、R4 均参与操作数选择。上述时序描述的是指令进入 E 时已经具有正确操作数，并非另行假定一条原图没有的同组组合前递。

六、第 4 题：增加精确异常支持

6.1 精确异常的状态要求

设发生异常的指令为 I_k。进入异常处理程序时必须满足：

1. 所有比 I_k 老的指令已经完成提交。
2. I_k 本身没有产生不可撤销的架构状态更新。
3. 所有比 I_k 年轻的指令都没有提交，其执行结果被丢弃。
4. 保存异常指令地址、异常原因以及需要时的故障虚拟地址，使处理程序能够正确处理和返回。

“检测到异常”与“正式进入异常处理”必须区分。错误路径上的取指异常也只能携带标志，不能立即陷入。

6.2 修改后的处理器结构

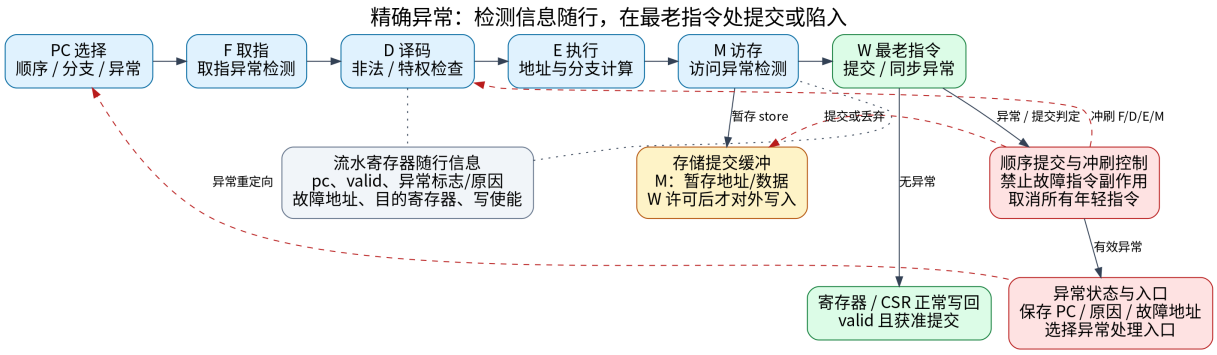


图 4: 第 4 题：增加异常随行信息、提交控制和存储缓冲后的五级流水线

图中 `pc/valid/exc` 信息沿流水寄存器随指令传递；W 级统一决定正常提交或异常入口。蓝色主链为流水线，红色虚线为冲刷和提交控制。

新增或修改的位置	必要逻辑
F 与 F/D	保存 PC、有效位；记录取指地址错、权限或 TLB 等异常信息
D 与 D/E	记录非法指令、特权违规；带上传递的异常，不产生真实副作用
E 与 E/M	按 ISA 记录需要检查的地址或运算异常；`addi.w` 本身不应凭空增加“溢出陷阱”
M 与 M/W	记录访存访问、对齐、地址翻译等异常；异常信息随指令传向 W
W 提交控制器	按程序顺序检查最老指令；正常则提交，异常则禁止该指令提交并进入异常处理
异常状态寄存器	保存异常返回地址、原因、故障地址；如采用 LoongArch，可对应 ERA、ESTAT、BADV 等，并按异常类型更新相关 CSR

新增或修改的位置	必要逻辑
PC 选择逻辑	在正常顺序、分支重定向、异常入口、异常返回之间选择；老指令异常优先于年轻指令重定向
valid/flush 控制	在异常或错误预测时清除被取消指令的有效位；所有写使能均与有效位及提交许可相与
存储提交缓冲	M 级保存 store 的地址、数据及检查结果，W 级确认无异常且允许提交后才使其对外生效

6.3 异常处理流程

1. 在可能发现异常的各级检测并记录原因，但不立即修改异常 CSR 或跳入处理程序。
2. 将异常与 PC、`valid` 一同传向 W。已有异常的指令禁止访问有副作用的资源。
3. 当异常指令成为 W 级最老指令时，更老的指令已经提交。
4. 禁止异常指令的寄存器、CSR 和内存写入；丢弃年轻的指令及存储缓冲项。
5. 保存异常上下文，清空流水线，将 PC 指向对应异常入口。

同一指令可能满足多种异常条件，优先级按 ISA 规定编码；不同指令同时报告异常时，必须先处理**程序顺序最老**的有效异常，而不是简单选择“最先被电路检测到”的异常。

分支预测恢复也要遵守年龄关系：老分支冲刷其错误路径上的年轻异常；老指令异常则冲刷年轻分支及其预测结果。

6.4 为什么 store 和中断需要单独处理

如果年轻的 store 已在 M 级不可撤销地修改内存，随后老指令在 W 抛出异常，就无法满足精确异常。因此本方案将 store 的对外写入推迟到提交许可之后；也可以采用能证明同等顺序约束的其他实现，但不能只给寄存器堆添加 `flush`。

MMIO 读写具有副作用时也必须受提交约束，不能把它们当作普通可撤销的 Cache 读。存储缓冲还需要通过旁路或停顿保证年轻 load 不会读到错误的旧值。

中断在指令提交边界接收，保存应继续执行的 PC。简化实现可停止新取指，排空在途指令并等待已提交的存储安全完成后再进入中断处理。同步异常保存故障指令 PC，中断保存恢复边界 PC，两者不能混为一谈。

本题给出的是逻辑结构与正确性分析，并未声称已经完成上述逻辑的 RTL 或形式化验证。

七、第 5 题：含前递的双发射五级静态流水线

双发射的“每周期最多发射 2 条”不是“任意相邻两条都可一起执行”。本程序具有以下约束：

- I 写 `t0`，D_1 读并写 `t0`，不能在同组 E 中使用跨周期前递解决相关。
- D_i 写 `t0`，B_i 立即读取它，也不能同组发射。
- 循环没有独立计算可填入另一路；默认顺序取指仍需等待 E 级分支结果重定向。

因此，在不支持“同组 ALU 输出到同组分支”的组合级联、没有循环展开的约定下，有效执行速度没有提升：

$$\boxed{C_5 = 403.}$$

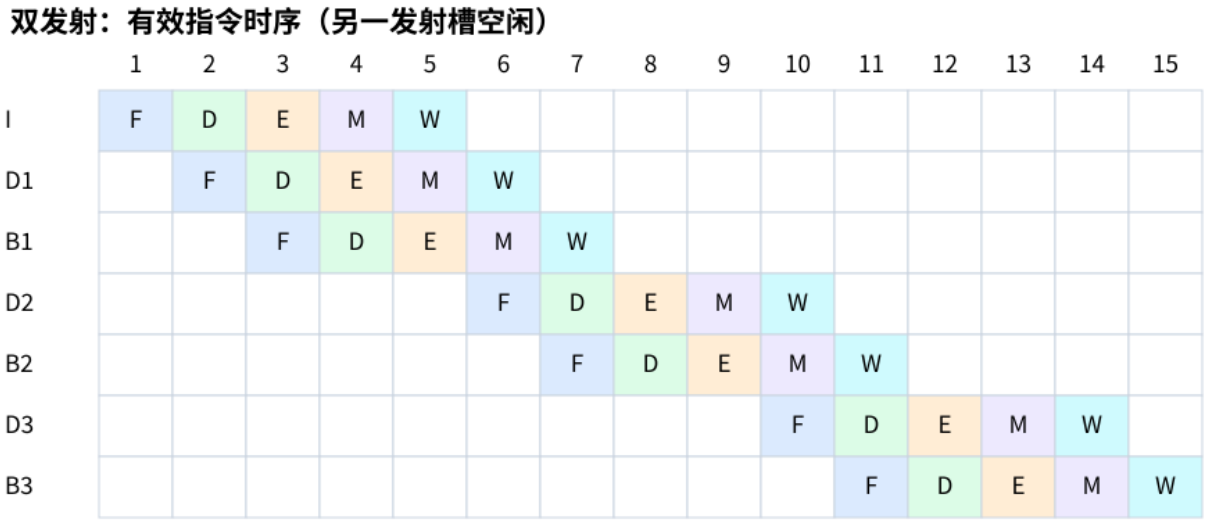


图 5: 第 5 题：双发射流水线有效指令时空图，另一发射槽空闲

图 9.18 中，一条指令因数据相关停顿时，同组另一条指令会陪同阻塞。它展示的是先后执行组之间的前递，不能据此让同组减一与分支同时执行。此程序按相关约束分开发射，另一槽位空闲，图中只列有效指令。因此不能直接按 $\lceil 201/2 \rceil + 4$ 计算。

八、第 6 题：数据相关与解决方法

8.1 三种寄存器数据相关

令 I_i 比 I_j 先执行。

类型	定义	示例
RAW，写后读，真相关	I_j 读取 I_i 写入的值	`add r1,r2,r3; sub r4,r1,r5`
WAR，读后写，反相关	I_j 不得在 I_i 读取旧值前覆盖该寄存器	`add r4,r1,r2; sub r1,r3,r5`
WAW，写后写，输出相关	两条指令写同名寄存器，最终值应来自年轻指令	`add r1,r2,r3; sub r1,r4,r5`

RAR，即两条指令都只读同一寄存器，不构成需要保持先后的数据相关。

8.2 静态流水线的处理

- **RAW：**在译码/发射处检测；结果已可用时前递，尚不可用时停顿并插入气泡。五级流水线的紧邻 load-use 通常仍需停顿，因为数据产生得更晚。
- **WAR：**简单顺序五级流水线按程序顺序在 D 读、在 W 写，年轻指令不会抢先覆盖老指令尚未读取的值，通常不会形成实际冒险。
- **WAW：**固定阶段、顺序写回使写入顺序与程序顺序一致，通常不会形成实际冒险。若引入不同执行时延或非顺序写回，则仍须增加约束。
- 双发射还需检测同组相关及功能单元冲突，不能只处理不同周期的指令。

8.3 带寄存器重命名的动态流水线

重命名让不同版本的同名架构寄存器映射到不同物理寄存器：

- WAR、WAW 是名字重复导致的假相关，可由重命名消除。
- RAW 体现真正的值依赖，重命名不能消除。发射队列/保留站按物理寄存器标签等待生产者，通过唤醒、选择和结果旁路执行。
- 执行可以乱序；重排序缓冲区保持顺序提交，配合映射表恢复处理错误预测和精确异常。

内存也可能存在 RAW/WAR/WAW 相关。寄存器重命名不能消除潜在的访存地址冲突，还需 load/store 队列、地址消歧和必要的 store-to-load 前递等机制。

九、第 7 题：D 级永远预测跳转

9.1 预测与验证流程

1. 分支在 D 被识别，算出目标并预测为跳转。
2. 下一周期取目标指令；因分支不能在 F 就完成目标选择，仍有 1 个周期的重定向间隙。

3. 分支在 E 用真实操作数验证。前 99 次方向正确；最后一次预测跳转但实际不跳转，冲刷年轻指令并恢复顺序 PC。

在本报告统一的“最后分支通过 W”口径下：

$$\boxed{C_7 = (201 + 4) + 99 \times 1 = 304.}$$

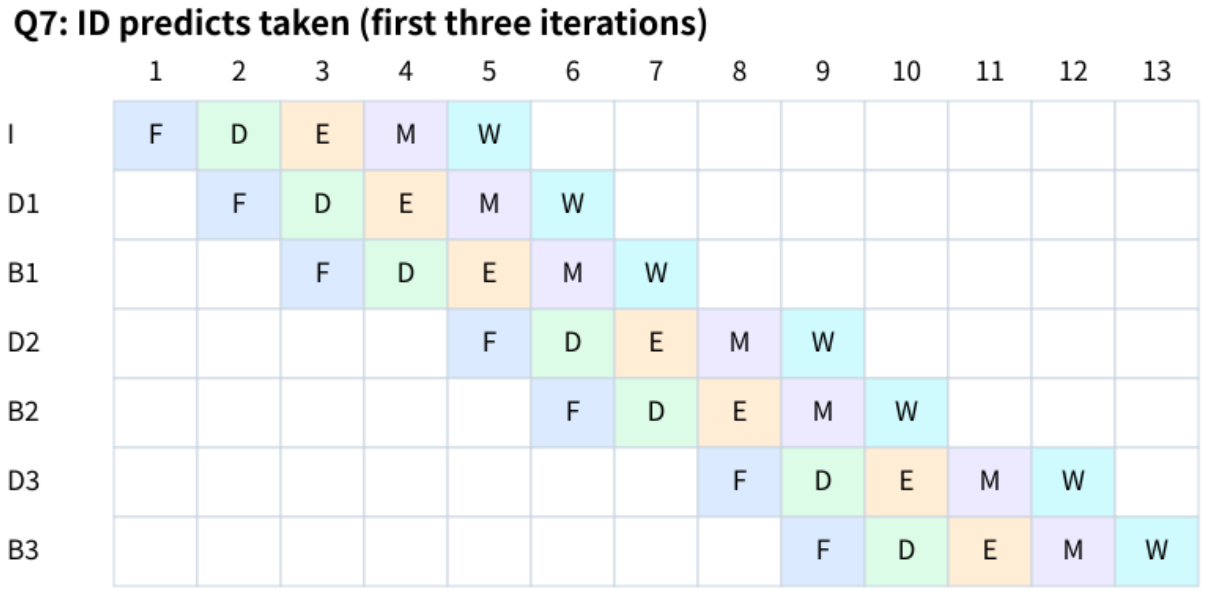


图 6: 第 7 题：D 级预测跳转的前三次循环时空图

B_1 的 D、E 分别为第 4、5 周期；D_2 第 5 周期 F。之后每次循环的有效启动间隔为 3 周期。

9.2 最后一次误判为什么不能机械再加罚时

末尾关键时刻是：

指令/事件	周期
D_{100} 的 F、D、E、M、W	299、300、301、302、303
B_{100} 的 F、D、E、M、W	300、301、302、303、304
错误预测目标 D_{101} 被取回	302
B_{100} 验证不跳转并取消错误路径	第 302 周期末
循环之后第一条正确指令最早取指	303

最后一次错误预测**确实发生了恢复**，脚本也统计到 1 次 E 级纠错。但程序段止于 B_{100}，它已经在流水线内，恢复不会让它自己的 W 从 304 变晚。因此不能在已按完整时空图计算出的 304

上，再机械加一次对“后续指令取指”的罚时。

若评价的是包含循环后续代码的完整程序，需要将那些指令放入流水线，实际计算其取指和完成时刻。本报告没有把未给出的指令加入动态指令数。

9.3 计时终点

本报告第 2、3、5、7 题统一保留分支的 M、W。若教材把分支在 E 视为完成，并以“所有有架构意义的结果已完成”为终点，则参考模型中相应时刻分别为 603、401、401、303；第 7 题还需等待 D_{100} 在第 303 周期写回。不能一题数到 E、另一题数到 W 却不说明口径。

第 7 题的预测器在 D 级给出目标，仍需区分预测重定向与 E 级实际判定，不能将其等同于 F 级已知目标的预测器。

十、第 8 题：三重循环的 BHT 预测准确率

原程序为：

```
for(i=0;i<10;i++)
    for(j=0;j<10;j++)
        for(k=0;k<10;k++)
            {...}
```

10.1 必须明确的统计约定

高级语言循环没有给出汇编分支位置和方向。主解采用常见的**循环底部条件回跳**模型：每执行一次十次循环，分支序列为 T^9N。仅统计控制这三重循环的条件分支，不统计循环体内部未知分支和无条件跳转。

假设三个静态条件分支使用各自独立、无冲突的 BHT 项，预测器不会在每次重新进入内层循环时清零。初始值均为 0，其中一位的 0 预测不跳转；两位计数器 00 为强不跳转。

循环	调用次数	每次分支次数	总分支次数
内层 k	100	10	1000
中层 j	10	10	100
外层 i	1	10	10
合计	111	—	1110

10.2 一位 BHT

每项保存上一次分支结果。初始预测 N，在每段 T^9N 中：

- 第一个 T：预测 N，错误。
- 中间 8 个 T：正确。
- 最后一个 N：预测 T，错误，并将状态改为 N。

因此下一次进入同一循环时仍然错 2 次，每次正确 8 次：

$$E_1 = 100 \times 2 + 10 \times 2 + 1 \times 2 = 222,$$

$$\boxed{A_1 = \frac{1110 - 222}{1110} = 80\%.}$$

10.3 两位饱和计数器 BHT

状态	预测	实际 T 后	实际 N 后
00，强 N	N	01	00
01，弱 N	N	10	00
10，弱 T	T	11	01
11，强 T	T	11	10

每个独立 BHT 项第一次遇到 T^9N 时，前两个 T 都预测错误，最后的 N 也错误，共 3 次。结束时状态为 10。以后每次从 10 开始，第一个 T 就能正确预测，只有退出时的 N 错误，每次 1 次。

因此，某一循环被调用 r 次时，误判次数为 $3 + (r - 1) = r + 2$：

$$E_2 = (100 + 2) + (10 + 2) + (1 + 2) = 117,$$

$$\boxed{A_2 = \frac{1110 - 117}{1110} = \frac{331}{370} \approx 89.4595\%.}$$

循环	一位误判	两位误判	两位准确率
内层	200	102	89.8%
中层	20	12	88%
外层	2	3	70%

循环	一位误判	两位误判	两位准确率
总计	222	117	89.4595%

总体准确率按动态分支次数加权，不能把三个循环的准确率直接取算术平均。两位的外层准确率反而较低，是因为只调用一次，冷启动误判还未被摊薄。

10.4 为什么有些答案会不同

若忽略初始化、把所有两位计数器都当成已预热为预测 T，则每段只错退出分支，得到近似 90%。本题明确初始值为 0，所以主解不能忽略前两个 T 的训练过程。

若保留循环顶部检查，并约定“条件成立则跳入循环体”，每次会得到 $T^{10}N$，共 $1100+110+11=1221$ 个条件分支。相同初始状态下，误判总数仍分别为 222 和 117：

$$A_1' = \frac{999}{1221} \approx 81.8182\%, \qquad A_2' = \frac{1104}{1221} \approx 90.4177\%.$$

这个替代模型也由脚本复核。如果顶部条件分支采用“条件不成立则跳出”的相反方向，状态序列还会改变。若 BHT 有地址冲突，三个分支也会互相干扰。因而高级语言题目的严格数值答案必须附上分支形式和 BHT 无别名假设。

十一、第 9 题：Cache 地址字段划分

已知物理地址 32 位、Cache 数据容量 32 KiB、块大小 64 B，按字节编址。

$$N_{\text{line}} = \frac{32 \times 1024}{64} = 512, \qquad b_{\text{offset}} = \log_2 64 = 6.$$

对于 w 路组相联：

$$N_{\text{set}} = 512/w, \quad b_{\text{index}} = \log_2(512/w), \quad b_{\text{tag}} = 32 - b_{\text{index}} - 6.$$

组织方式	路数	组数	Tag 位数	Index 位数	Offset 位数
直接映射	1	512	17	9	6
两路组相联	2	256	18	8	6
四路组相联	4	128	19	7	6

32 位地址具体划分：

直接映射： `[ Tag:31..15 ][ Index:14..6 ][ Offset:5..0 ]`
两路组相联： `[ Tag:31..14 ][ Index:13..6 ][ Offset:5..0 ]`
四路组相联： `[ Tag:31..13 ][ Index:12..6 ][ Offset:5..0 ]`

路号由同一组内多个 Tag 的比较结果确定，不需要再从地址中单独划出“路号位”。32 KiB 按数据容量理解，不含 Tag、有效位、脏位和替换状态开销。

十二、第 10 题：Cache 命中率与主频的性能权衡

两方案动态指令数相同。记不含数据 Cache 不命中停顿的基础 CPI 为 C_0，方案一主频为 f_1，则方案二主频为 $f_2 = 0.9 f_1$。

load/store 占比为 0.4，每次不命中增加 100 个本机时钟周期：

$$CPI_1 = C_0 + 0.4(1 - 0.85) \times 100 = C_0 + 6,$$

$$CPI_2 = C_0 + 0.4(1 - 0.95) \times 100 = C_0 + 2.$$

不能只比较命中率或 CPI，还必须使用：

$$T = \frac{IC \times CPI}{f}, \qquad \frac{T_2}{T_1} = \frac{C_0 + 2}{0.9(C_0 + 6)}.$$

若按常用的理想基础 $C_0 = 1$ 计算，则：

$$CPI_1 = 7, \quad CPI_2 = 3, \quad \frac{T_2}{T_1} = \frac{3}{0.9 \times 7} = \frac{10}{21} \approx 0.47619.$$

方案二更快，性能为方案一的2.1 倍，执行时间减少约52.38%.

缺失参数的边界：题目没有明确提供基础 CPI。严格地说，无条件优劣判断需要这个信息。求解 $T_2 < T_1$ 得：

$$C_0 + 2 < 0.9(C_0 + 6) \iff C_0 < 34.$$

所以 $C_0 < 34$ 时方案二更优，$C_0 = 34$ 时相同，$C_0 > 34$ 时方案一更优。常见五级标量流水线的理想 $C_0 = 1$ 属于方案二占优的范围。

十三、实验复核与结果汇总

13.1 可复现方法

本目录包含：

- `verify.py`：仅依赖 Python 标准库的流水线和 BHT 验证脚本。
- `results.json`：本次运行产生的周期数、前缀时序和分支预测统计。
- `figures/`：可编辑的 SVG 时空图、异常结构图源文件及用于报告排版的 PNG。
- `build.sh`：重跑验证、生成图片并编译本报告 PDF 的命令。

从仓库根目录运行：

```
python3 homework/体系结构习题报告/verify.py
```

生成 PDF 则运行：

```
bash homework/体系结构习题报告/build.sh
```

PDF 构建需要 Pandoc、XeLaTeX、ImageMagick、Graphviz 和报告头部指定的字体。只复核数值不需要这些排版工具。

13.2 本次执行的有效检查

1. 对循环上界 1—30，分别实际运行无前递、有前递、D 级预测跳转三种逐周期模型，共 90 次；验证动态执行条数、最终 `t0=0`、跳转次数和周期公式。
2. 对上界 100 运行同样三种模型，分别得到 605、403、304 周期；错误预测模型确实在最后一次分支纠错，而不是省略了最终恢复。
3. 双发射图检查同组依赖不允许同时进入 E；403 周期由相关约束和单发射有效时序推导，未声称建立了完整通用双发射 CPU 模拟器。
4. 逐次更新一位/两位 BHT，独立统计三层循环及两种分支放置方式，保留全零冷启动影响。
5. 检查各 Cache 划分满足 组数 $\times$ 路数 $\times$ $64 = 32768$ 且地址位宽之和为 32。
6. 使用精确分数复核性能比 $10/21$ 和 2.1 倍加速比。

多周期时序依据图 9.4 的五拍控制检查为 1005 周期。流水线脚本用于验证推导和有效指令时序；精确异常设计未进行 RTL 测试。

13.3 答案汇总

题号	结果或要点	适用条件
1	1005 周期	图 9.4 五拍固定周期控制

题号	结果或要点	适用条件
2	605 周期	无前递、W/D 同周期先写后读、E 判分支、末分支过 W
3	403 周期	向 E 级 ALU/分支前递、E 判分支、末分支过 W
4	异常随行，W 按序提交；延迟存储副作用；冲刷年轻指令	逻辑设计，非已实现 RTL
5	403 周期	同组 RAW 不能发射；无指令重排或组内组合级联
6	RAW 需等待/前递；WAR、WAW 可通过重命名消除	内存相关仍需单独处理
7	304 周期	D 预测、E 验证、末分支过 W，程序段止于循环分支
8	一位 80%；两位约 89.4595%	底部 T^9N、独立 BHT 项、初始全 0
9	Tag/Index/Offset：17/9/6、18/8/6、19/7/6	32 位字节地址、数据容量 32 KiB
10	基础 CPI=1 时方案二快 2.1 倍	一般条件为基础 CPI 小于 34

报告中的时序图、数值结果和脚本采用相同的计时终点，便于逐项复核。